

리눅스 3 - 데이터베이스 페이지와 RAM의 관계

남궁명수

핵심

데이터베이스의 데이터는 영구적으로 SSD/HDD의 데이터 파일에 저장되고,
DBMS는 필요한 데이터베이스 페이지를 RAM의 버퍼 풀로 가져온 뒤 CPU가 처리한다.

CPU는 SSD/HDD의 레코드를 직접 읽을 수 없다.
CPU가 실제로 연산하려면 데이터가 RAM에 올라와 있어야 한다.

SSD/HDD의 데이터 파일
↓ 페이지 단위로 읽기
RAM의 버퍼 풀
↓ 메모리 접근
CPU가 검색·비교·연산

[1] 데이터베이스 페이지란?

데이터베이스 페이지는 DBMS가 데이터를 저장하고 읽어오는 기본 단위다.

테이블의 레코드나 B+Tree 인덱스는 파일 안에 아무렇게나 저장되는 것이 아니라,
일정한 크기의 페이지로 나뉘어 저장된다.

[데이터베이스 파일]



각 페이지에는 대략 다음 정보가 들어간다.

- 페이지 번호와 종류
- 페이지 관리 정보
- 인덱스 키
- 레코드
- 빈 공간
- 다른 페이지와의 연결 정보

DBMS에 따라 페이지 크기는 다르다.

예를 들어, MySQL InnoDB에서는 일반적으로 16KiB 페이지를 사용한다.

중요한 점은 DBMS가 보통 레코드 하나만 SSD에서 가져오는 것이 아니라, 그 레코드가 들어 있는 페이지 전체를 가져온다는 것이다.

[2] 데이터베이스 페이지는 어디에 저장되는가?

동일한 데이터베이스 페이지가 상황에 따라 두 장소에 존재할 수 있다.

위치	역할
SSD/HDD의 데이터 파일	데이터를 영구적으로 보관
RAM의 버퍼 풀	현재 사용 중이거나 다시 사용할 가능성이 있는 페이지를 임시 보관

SSD/HDD에 있는 것이 데이터의 영구적인 원본이고, RAM의 버퍼 풀에는 작업을 위한 페이지 사본이 올라온다.

SSD/HDD		RAM
데이터 파일		버퍼 풀
페이지 10	—필요할 때 읽기—>	페이지 10의 사본
페이지 11		페이지 25의 사본
페이지 12		페이지 37의 사본
...		

RAM은 전원이 꺼지면 내용이 사라지므로 영구 저장소가 아니다.

[3] 버퍼 풀이란?

버퍼 풀은 DBMS가 데이터베이스 페이지를 보관하기 위해 **RAM에 확보한 메모리 공간**이다.

DBMS는 쿼리를 처리할 때 먼저 다음을 확인한다.

필요한 페이지가 버퍼 풀에 있는가?

[버퍼 풀에 있는 경우: 버퍼 히트]

CPU

↓

RAM의 버퍼 풀에서 바로 접근

SSD/HDD를 읽지 않아도 되므로 빠르다.

[버퍼 풀에 없는 경우: 버퍼 미스]

1. DBMS가 필요한 페이지를 찾는다.
2. 운영체제에 저장장치 읽기를 요청한다.
3. SSD/HDD에서 페이지를 읽는다.
4. 페이지를 RAM의 버퍼 풀에 올린다.
5. CPU가 버퍼 풀의 페이지를 처리한다.

저장장치 I/O가 발생하므로 버퍼 히트보다 느리다.

[4] 실제 레코드는 어떻게 찾는가?

다음 테이블이 있다고 하자.

id	name	age
101	철수	25
102	영희	30
103	민수	28

그리고 age에 B+Tree 인덱스가 있다고 하자.

```
SELECT * FROM member WHERE age = 28;
```

DBMS는 대략 다음 순서로 접근한다.

[1단계] 루트 페이지 접근

B+Tree의 루트 노드도 데이터베이스 페이지에 저장되어 있다.

루트 페이지
[20 | 40 | 60]

DBMS는 28이 20~40 범위에 있다는 것을 확인하고 해당 자식 페이지 번호를 알아낸다.

$20 \leq 28 < 40$
↓
페이지 105로 이동

[2단계] 자식 페이지 접근

DBMS는 페이지 105가 RAM의 버퍼 풀에 있는지 확인한다.

페이지 105가 버퍼 풀에 있음?

- 있으면 RAM에서 바로 읽는다.
- 없으면 SSD/HDD에서 페이지 105 전체를 읽어 버퍼 풀에 올린다.

[3단계] 리프 페이지에서 인덱스 키 발견

보조 인덱스라면 리프 페이지에는 대략 다음 정보가 들어 있다.

[인덱스 키: 28 | 기본키: 103]

age=28에 해당하는 레코드의 기본키가 103이라는 것을 알아낸다.

[4단계] 기본키 인덱스 접근

InnoDB와 같은 클러스터형 저장 방식에서는 기본 B+Tree를 다시 탐색한다.

기본키 103 검색

↓

기본키 인덱스의 리프 페이지

↓

[103 | 민수 | 28]

이 페이지도 버퍼 풀에 없으면 SSD/HDD에서 페이지 단위로 읽어와야 한다.

[5] 왜 레코드 하나가 아니라 페이지 전체를 읽는가?

SSD/HDD는 매우 작은 데이터를 여러 번 읽는 것보다, 일정한 크기의 묶음을 한 번에 읽는 것이 효율적이다.

또한 서로 가까운 레코드는 함께 사용될 가능성이 높다.

예를 들어, 페이지 하나에 다음 레코드들이 들어 있다고 해보자.

[페이지 30]

[101 | 철수 | 25]

[102 | 영희 | 30]

[103 | 민수 | 28]

[104 | 지수 | 26]

103번은 레코드를 찾기 위해 페이지 30을 읽으면, 같은 페이지의 101, 102, 104도 함께 RAM으로 들어온다.

그 다음 가까운 레코드를 조회하면 저장장치를 다시 읽지 않고 RAM에서 처리할 가능성이 높다.

이를 **공간 지역성**이라고 한다.

[6] 데이터를 수정하면 어떻게 되는가?

다음 쿼리를 실행한다고 하자.

```
UPDATE member  
SET age = 29  
WHERE id = 103;
```

DBMS는 일반적으로 SSD/HDD의 데이터 페이지를 그 자리에서 즉시 직접 수정하지 않는다.

1. 필요한 페이지를 버퍼 풀에 가져온다.
2. RAM에 있는 페이지를 수정한다.
3. 수정된 페이지를 Dirty Page로 표시한다.
4. 나중에 저장장치의 데이터 파일에 기록한다.

SSD/HDD 원본 페이지

↓ 읽기

RAM 버퍼 풀의 페이지

↓ age 28 → 29 수정

Dirty Page

↓ 나중에 기록

SSD/HDD 데이터 파일 갱신

[Dirty Page]

RAM에서 수정되었지만 아직 데이터 파일에 반영되지 않은 페이지를 Dirty Page라고 한다.

반대로 RAM의 내용과 저장장치의 내용이 같은 페이지를 Clean Page라고 한다.

상태	의미
Clean Page	RAM과 저장장치의 내용이 동일
Dirty Page	RAM에서 수정됐지만 데이터 파일에는 아직 완전히 반영되지 않음

[7] RAM에서만 수정하면 장애가 났을 때 사라지지 않는가?

그 위험을 해결하기 위해 DBMS는 로그를 사용한다. 대표적인 방식이 WAL(Write-Ahead Logging)이다.

실제 데이터 페이지를 저장장치에 기록하기 전에 변경 내용을 로그에 먼저 기록한다.

UPDATE 실행

↓

로그 버퍼에 변경 내용 기록

↓

로그를 저장장치에 먼저 기록

↓

트랜잭션 COMMIT

↓

Dirty Page는 이후 데이터 파일에 반영

장애가 발생하더라도 저장장치에 기록된 로그를 이용해 변경 내용을 복구할 수 있다.

따라서 COMMIT이 항상 “수정된 데이터 페이지 전체가 즉시 데이터 파일에 저장되었다”는 뜻은 아니다.

일반적으로는:

- 복구에 필요한 로그가 안전하게 기록되고
- 실제 Dirty Page는 이후 적절한 시점에 데이터 파일로 기록될 수 있다.

[WAL 로그 VS 원본 데이터]

[1] 속도의 차이(순차 쓰기 vs 무작위 쓰기)

- WAL은 순차 쓰기(Sequential I/O): 로그 파일은 기존 파일 내용 뒤에 단순히 이어붙이기만 하면된다 디스크 바늘이 움직이지 않고 한 방향으로 쪽 쓰기 때문에 속도가 매우 빠르다.
- 원본은 무작위 쓰기(Random I/O): 원본 데이터 파일은 하드디스크 이곳저곳에 흩어져 있다. 1건을 수정하려면 디스크의 전혀 다른 위치들을 찾아다니며 수정해야 하므로 시간이 수십~수백 배 더 걸린다.

[2] 안전성의 차이 (버퍼 풀과 데이터 유실)

데이터베이스는 성능을 위해 모든 데이터를 메모리(Buffer Pool)에 올려두고 작업한다.

- 수정 즉시 원본 디스크에 쓰면: 사용자가 ‘저장’을 누를 때마다 느린 디스크 무작위 쓰기가 끝날 때까지 한참을 기다려야 한다. 서비스가 엄청나게 느려진다.
- 메모리에만 두고 나중에 쓰면: 속도는 빠르지만, 디스크에 쓰기 전에 컴퓨터 전원이 꺼지면 메모리에 있던 데이터가 전부 증발한다.
- WAL을 사용하면: 메모리에서 수정한 내용을 가장 빠른 순차 쓰기로 WAL 파일에 저장을 끝낸 뒤 사용자에게 완료 통보를 한다. 그 후 원본 파일은 한가할 때 배경에서 천천히 업데이트(Checkpoint)한다. 중간에 전원이 꺼져도 WAL 파일에 기록이 남아 있음으로 100% 복구할 수 있다.

[8] 버퍼 풀이 가득차면 어떻게 되는가?

RAM의 버퍼 풀은 크기가 제한되어 있다.

새로운 페이지를 가져올 공간이 부족하면 DBMS는 기존 페이지 중 일부를 내보내야 한다.

새로운 페이지 필요
↓
버퍼 풀에 빈 공간 없음
↓
교체할 페이지 선택
↓
기존 페이지 제거
↓
새 페이지 적재

Clean Page라면

저장장치와 내용이 같으므로 버퍼 풀에서 바로 제거할 수 있다. 필요해지면 저장장치에서 다시 읽으면 된다.

Dirty Page라면

수정 내용이 RAM에만 있으므로 먼저 데이터 파일에 기록한 후 제거해야 한다.

[9] DB 페이지와 운영체제의 메모리 페이지는 다르다

여기서 혼동하면 안되는 개념이 있다.

구분	데이터베이스 페이지	운영체제 메모리 페이지
관리 주체	DBMS	운영체제와 MMU
주요 목적	레코드와 인덱스 저장 및 I/O	가상 주소와 물리 메모리 관리
크기 예시	DBMS 설정에 따라 다름	흔히 4KiB
저장 위치	데이터 파일 및 버퍼 풀	RAM, 필요하면 스왑 등

예를 들어, 데이터베이스 페이지가 16KiB이고 운영체제 메모리 페이지가 4KiB라면,

DB 페이지 하나가 RAM에서 여러 OS 페이지에 걸쳐 존재할 수 있다.

[데이터베이스 페이지 16KiB]

OS 4KiB	OS 4KiB	OS 4KiB	OS 4KiB
---------	---------	---------	---------

즉,

- DB 페이지는 DBMS의 데이터 관리 단위
 - OS 페이지는 가상 메모리와 물리메모리 관리 단위
- 둘 다 '페이지'라고 부르지만 서로 다른 계층의 개념이다.

[10] OS페이지 캐시와 DB 버퍼 풀의 관계

파일을 읽으면 운영체제가 페이지 캐시에 데이터를 보관할 수도 있다.
그러면 다음처럼 데이터가 중복될 가능성이 있다.

SSD/HDD

↓

OS 페이지 캐시

↓

DBMS 버퍼 풀

일부 DBMS는 운영체제 페이지 캐시를 활용하고, 일부는 직접 I/O 방식을 사용해 페이지 캐시의 영향을 줄이기도 한다. 하지만 어떤 방식을 사용하든 중요한 흐름은 같다.

저장장치의 데이터는 결국 RAM에 올라와야 CPU가 처리할 수 있다.